Lecture 11

Cloud Deployment and Delivery to Users

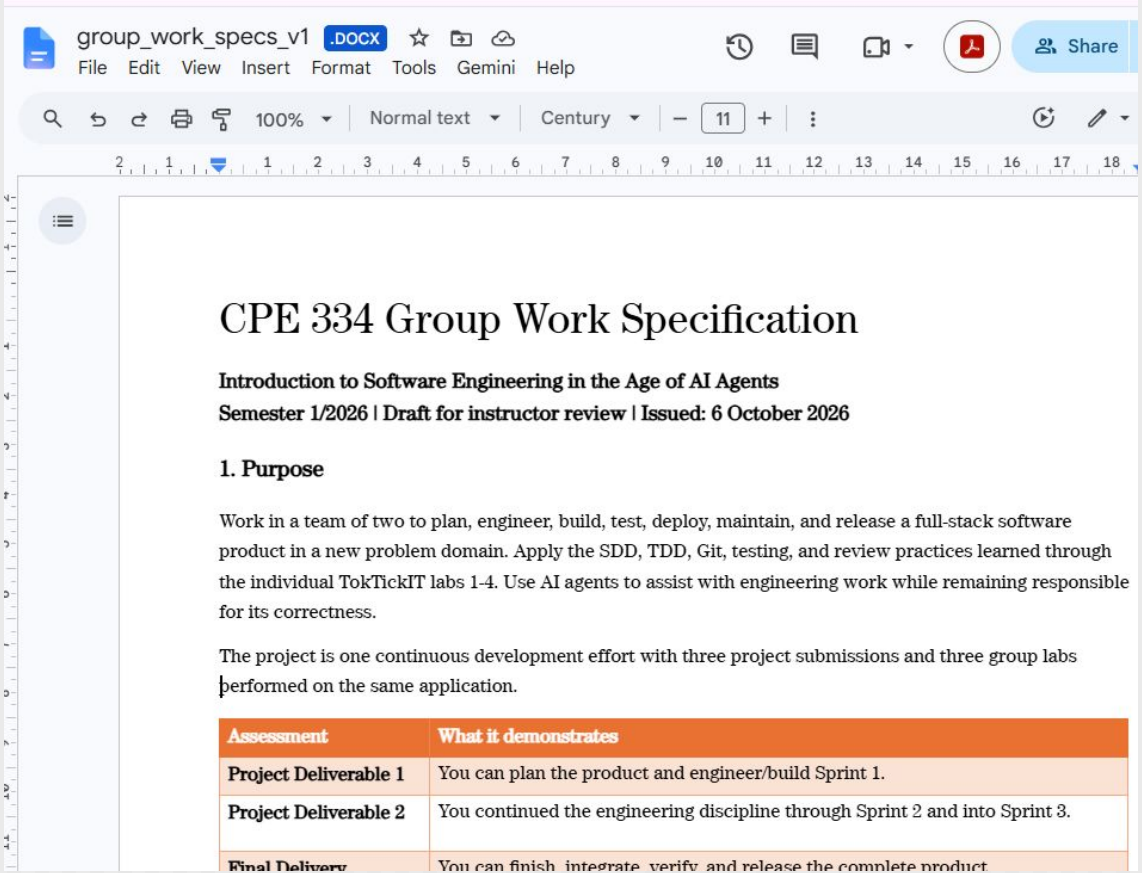
Lab 5: CI/CD, Docker, GCP deployment

Suthep “Jogie” Madarasmi, Ph.D.

Where we are

Week	Date	Topics	Activities
10	~~13, 15 Oct~~ 6, 8 Oct	Midterm Review and Individual Practical Integration **(Dr. Suthep)**	Lab: Individual Practical Challenge & Demo (midterm integration); begin Team Sprint 1 — working vertical slice
	14–28 Oct	Midterm Exam (covers Weeks 1–7)	
11	3, 5 Nov	Cloud Deployment and Delivery to Users **(Dr. Suthep)**	Lab: Cloud Deployment Lab — deploy the team application to the cloud (GCP) and deliver to users; Team Sprint 2
12	10, 12 Nov	Quality Assurance, UAT, Defect Management, Maintenance, and Technical Debt **(Dr. Piyanit)**	Lab: QA & UAT — write UAT scripts, defect triage & fixing, maintenance tickets, and a technical-debt log; Team Sprint 3
13	17, 19 Nov	Metrics, Reviews, Retrospectives, Ethics, Responsible AI Use, and AI Usage Report **(Dr. Suthep)**	Lab: Metrics, code review & retrospective; Responsible-AI reflection and AI Usage Report; final project hardening.
14	24, 26 Nov	Final Delivery, Product Demonstration, and Course Integration	Lab: Final Team Demo & Submission — product demonstration and course integration
	1–11 Dec	Final Exam (covers Weeks 8–14)	
	After Finals	Project Demonstration and Presentation	

Group Work Specs File



Points Distribution

Category / assessment	Raw points	Course Grade %
Individual Labs 1–4, existing aggregate	400	20%
Group Lab 5: CI/CD, Docker, and GCP deployment	100	5%
Group Lab 6: Cross-team UAT and defect management	100	5%
Group Lab 7: Maintenance, metrics, and release readiness	100	5%
All labs combined	**700**	**35%**
Project Deliverable 1	80	8%
Project Deliverable 2	70	7%
Final Project Delivery	100	10%
Group project subtotal	**250**	**25%**

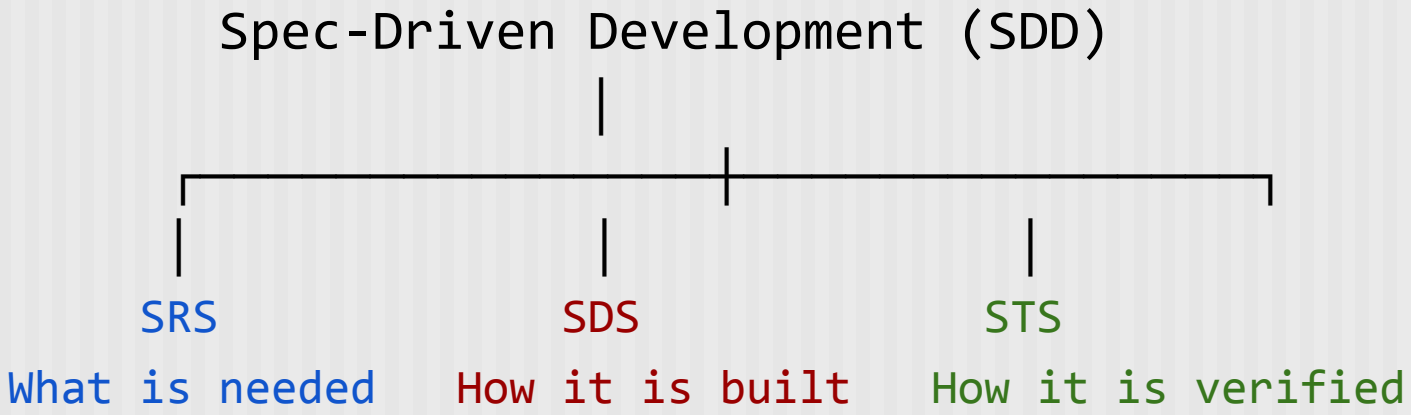
Each group work deliverable has its **own spec sheet** :

- Project Deliverables 1, 2, Final
- Lab 5, Lab 6, Lab 7

Deliverables and Schedule

Date	Activity / milestone	Points
6, 8 Oct	Miterm Exam Review Project launch: topics, scope, team workflow, engineering baseline	
12 - 25 Oct	Exam period; plan around your examination commitments	
26 Oct - 1 Nov	Midterm break; independent project work	
3, 5 Nov	Workshop: SRS, system SDS, Sprint 1 contract, tests, integration Lab 5: CI/CD, Docker, GCP deployment.	
8 Nov Sunday	Project Deliverable 1 due: baseline + working Sprint 1	80 project
10, 12 Nov	Lab 6: cross-team UAT and defect management	
15 Nov Sunday	Lab 5 due: deployed Sprint 1 and repeatable delivery	100 lab
17, 19 Nov	Lab 7: maintenance, metrics, release readiness	
22 Nov Sunday	Lab 6 due	100 lab
24, 26 Nov	Final Exam Review. Project Deliverable 2 Discussion.	
29 Nov Sunday	Deliverable 2 due Lab 7 due	70 project 100 lab
1 - 11 Dec	Final examination period	
13 Dec Sunday	Final product, report, and demo video due	100 project

Spec-Driven Development (SDD) for Coding Agents



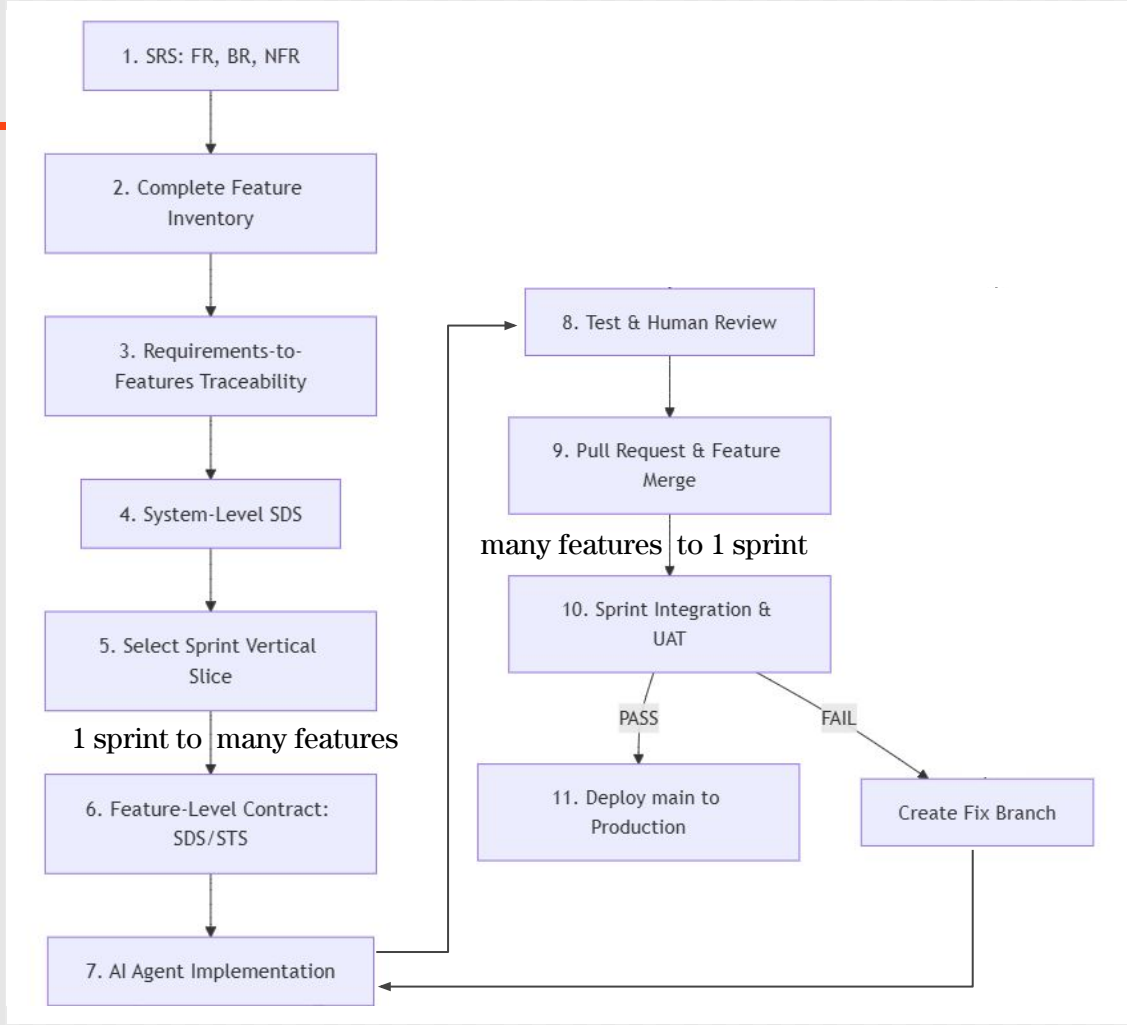
SRS - software requirements specification

SDS - Software Design Specification

STS - Software Test Specification

Spec Driven Dev

SDD



Spec Driven Development

1. **SRS**. FR, BR, NFR.
↓
2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...
↓
3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.
↓
4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment
↓
5. **Select the Sprint Vertical Slice.** Choose a "Feature Bundle" to be implemented in the sprint
↓

loop per feature

6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS
↓
7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications
↓
8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.
↓
9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch
↓
10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.
↓
11. **Per Release: Deploy main Branch to Production**

Where are we in the software lifecycle?



Already learned

SRS, SDS, STS, Git, PRs, tests and human review.

Today

Containers, cloud services, automated deployment and verification.

Two hours of concepts + two hours of practice

1. Cloud Platforms for Software Engineers

Why cloud?

Which kinds of services exist?

How would a company move a real application to cloud infrastructure?

2. Understand the Lab 5 machinery

How do GitHub, Docker, CI/CD, Cloud Run, Cloud SQL, secrets and deployment fit into one delivery system?

TA practical: configure and deploy TokTickIT on GCP

PART A

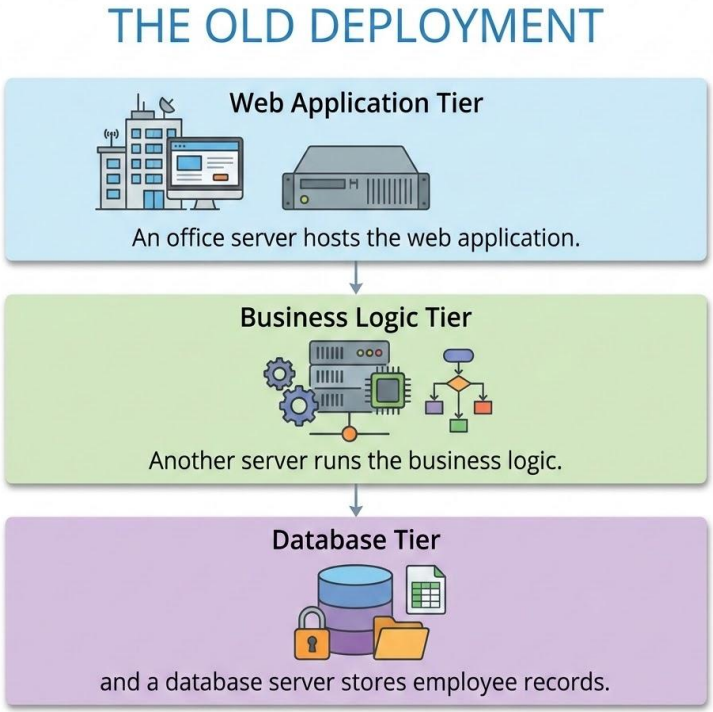
Cloud Platforms for Software Engineering

A real story: the company payroll system

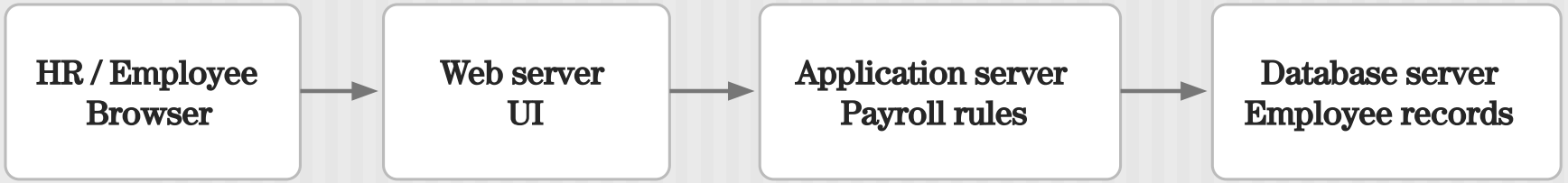
The application

1. **Data Input:** HR staff enter salaries and leave.
2. **Logic:** Payroll rules calculate deductions.
3. **Employee Access:** Employees download payslips.

Security Requirement:
Payroll records must remain private.



Traditional three-tier payroll architecture



Company IT buys and maintains the servers, OS, backups, networking and capacity.

Question: What happens if payroll traffic spikes on payday?

Why might the payroll company move to cloud?

Faster provisioning

Need a new server? Request a cloud resource in minutes.

Elastic capacity

Use more application instances during heavy traffic.

Managed components

Provider operates parts of the database and infrastructure.

Tradeoffs

Recurring bills, vendor dependence, outages and shared security responsibility.

Cloud does not mean “no servers”

Cloud computing: renting computing resources and managed services over a network.

Traditional: company owns machines

Cloud: provider owns infrastructure

Someone still operates the hardware. The key change is who provisions, manages and pays for which layers.

Public, private, hybrid, multicloud: four deployment choices

Public cloud

Payroll app runs on AWS, Azure or GCP infrastructure shared securely among customers.

Private cloud

A dedicated cloud environment managed for one organization.

Hybrid cloud

Keep HR records on premises while running a web portal in a public cloud.

Multicloud

An organization uses services from more than one cloud provider.

IaaS vs PaaS vs SaaS: who manages what?

IaaS infra	**Rent a virtual machine**	Compute Engine	We manage OS + payroll app
PaaS platform	**Deploy code or container**	Cloud Run	We manage payroll app + settings
SaaS software	**Use complete software**	Google Workspace	We manage user settings

"Serverless" means the provider handles most server provisioning and scaling; servers still exist.

Payroll: three ways to buy the same business capability

Infrastructure as a service

IaaS: Rent VMs

Install the payroll web server, backend, and DB on cloud VMs.

Management Focus

IT patches and operates all software and OS components.

Platform as a service

PaaS: Deploy App

Put payroll in Cloud Run and use managed Cloud SQL.

Management Focus

IT focuses primary attention on application development and logic.

Software as a service

SaaS: Subscribe

Use a ready-made payroll product directly off-the-shelf.

Management Focus

Company configures users and payroll rules instead of writing software.

Different brand names, similar engineering needs

Amazon Web Services

AWS ECOSYSTEM

- EC2 (Compute)
- ECS (Containers)
- RDS (Database)

Microsoft Azure

AZURE ECOSYSTEM

- Virtual Machines
- Container Apps
- Azure SQL

Google Cloud

GCP ECOSYSTEM

- Compute Engine
- Cloud Run
- Cloud SQL

Others

SPECIALIZED CLOUDS

- Huawei Cloud
- Oracle Cloud
- Alibaba Cloud
- DigitalOcean

Different names, same core questions: where does code run, where does data live, who can access it, how do releases happen?

The nine cloud service families a SE should recognize



Compute
Run code



Database
Save records



Object storage
Keep files



Networking
Connect services



Security
Control access



Containers
Package apps



CI/CD
Deliver versions



Monitoring
See problems



AI services
Use models

Payroll use case: three very different storage needs

developer_board

COMPUTE

Cloud Run

Executes requests: “calculate monthly payroll” and “generate payslip”.

dns

DATABASE

Cloud SQL

Stores employee IDs, salary history and payroll transactions.

folder_open

OBJECT STORAGE

Cloud Storage

Stores PDFs of payslips and related uploaded documents.

Key Question: Why not store payroll records only inside the application container?

What else does a production payroll app need?

Network + Load Balancing

Route employee requests to available application instances.

Security + Secrets

Give HR role access; keep database passwords out of code.

Monitoring + Logging

Find errors after payroll is released; alert on failures.

Messaging / Jobs

Generate and email many payslips asynchronously.

Two ways AI appears in software engineering

AI helps BUILD software

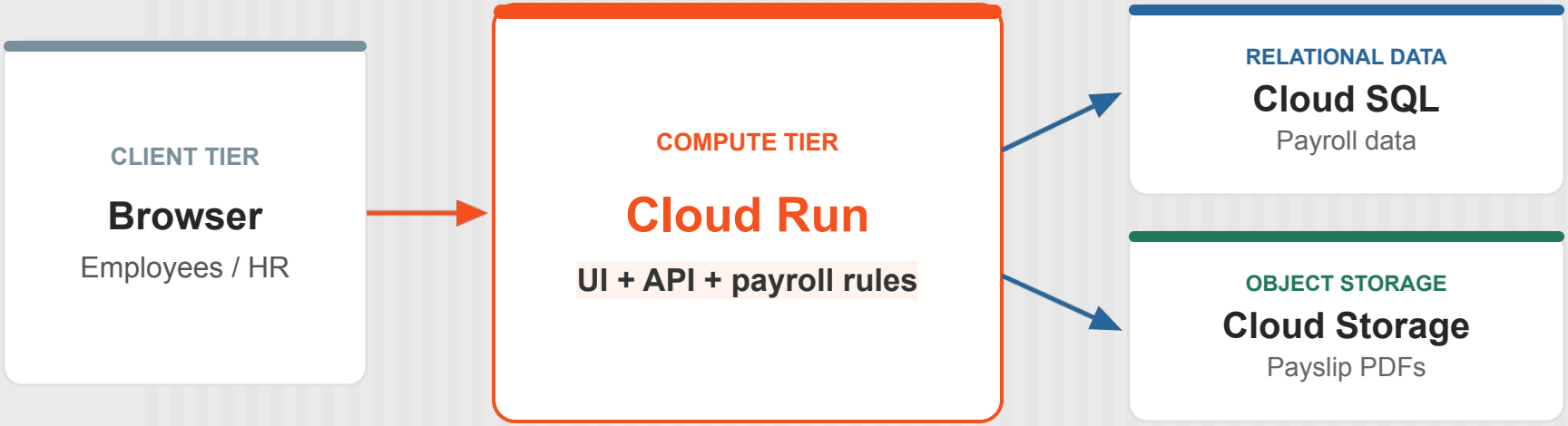
A coding assistant proposes a payroll API, writes tests, reviews a Dockerfile and helps debug CI. Human engineers remain accountable.

AI is PART OF the product

The payroll app calls a hosted LLM to explain payslip components or let HR search policy documents.

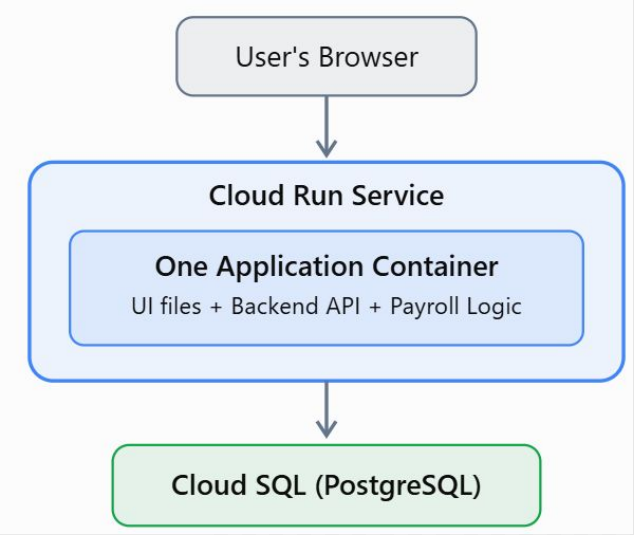
AI coding agent ≠ hosted AI service used by customers

Move the three-tier payroll system to GCP

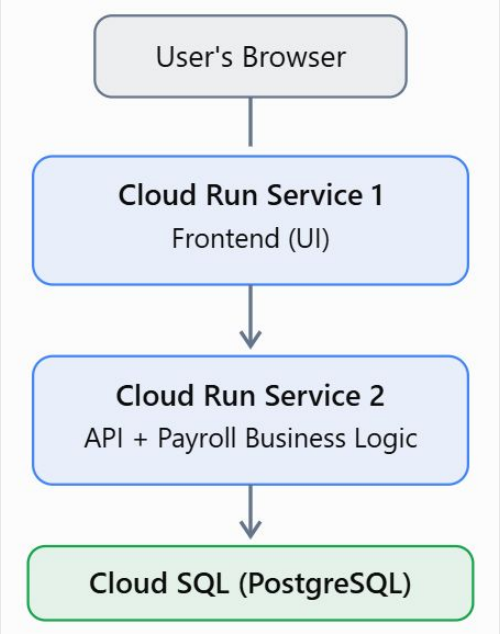


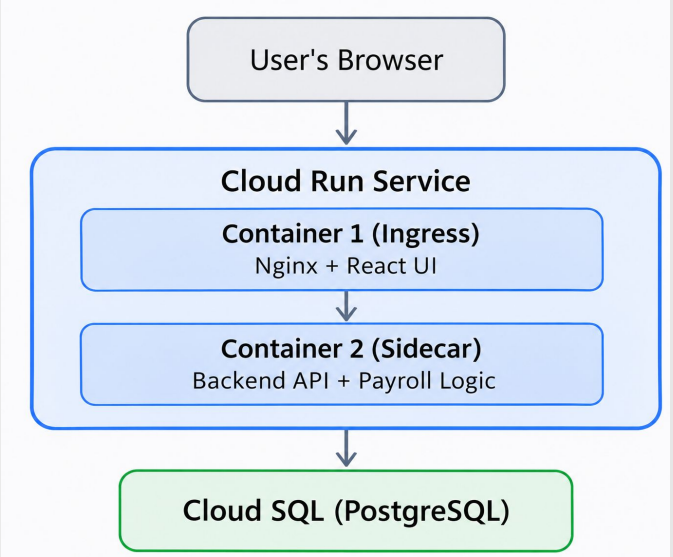
Logical layers can remain separate even when UI and API share one deployment container.

Two GCP Deployment Options



Lab 5 uses this option





Can PostgreSQL Live in a Container?

Short Answer

A container can run PostgreSQL.

What must survive is the **data**, not the container itself.

Writable Layer Risk: If data is only inside the container writable layer, it may disappear when replaced.

Valid Architecture: If data is stored on a persistent volume or managed storage, containerized DB works.

Payroll Example

1. Operational Scenario

HR enters payroll for 500 employees.

2. Ephemeral Storage Danger

If those records are stored only inside the app container, a redeploy can wipe them out completely.

3. Recommended Setup

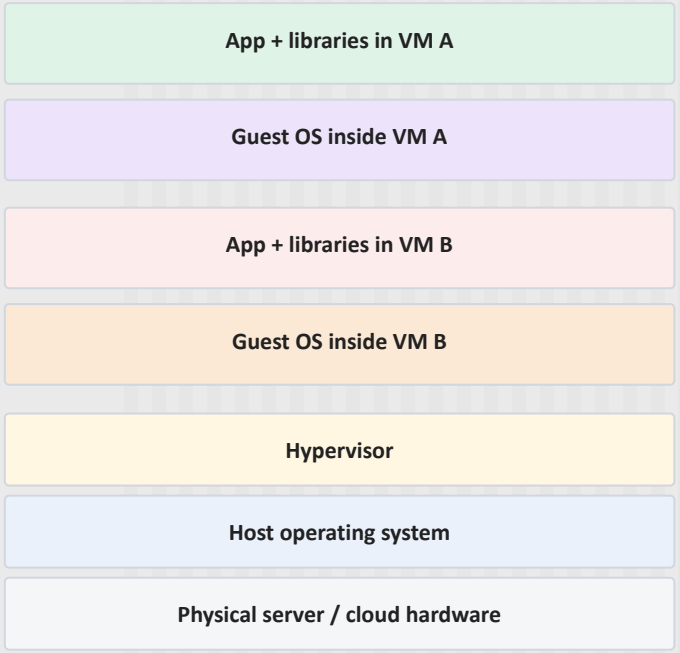
The payroll app may run in a container, but payroll data should live in Cloud SQL or persistent DB.

Deployment	Persistent?	Suitable for PSQL?	Comment
Docker on a VM with no persistent volume	No	No	Data can be lost if the container is replaced
Docker on a VM with a persistent disk / volume	Yes	Yes	A valid architecture when storage is managed correctly
Docker on Cloud Run using local container storage	No	No	Cloud Run filesystem is ephemeral
Cloud Run application + Cloud SQL	Yes	Yes (recommended)	Best match for our lab architecture
PostgreSQL on GKE with persistent volumes	Yes	Yes	Works, but is more complex than we need

Key Takeaway: Containers can be replaced. Persistent data must survive their replacement.

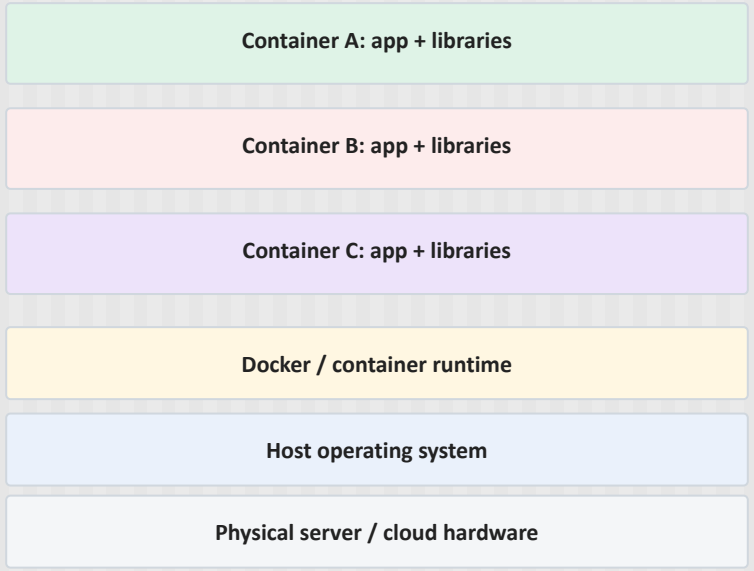
Containers vs Operating Systems vs Virtual Machines

Virtual machine model



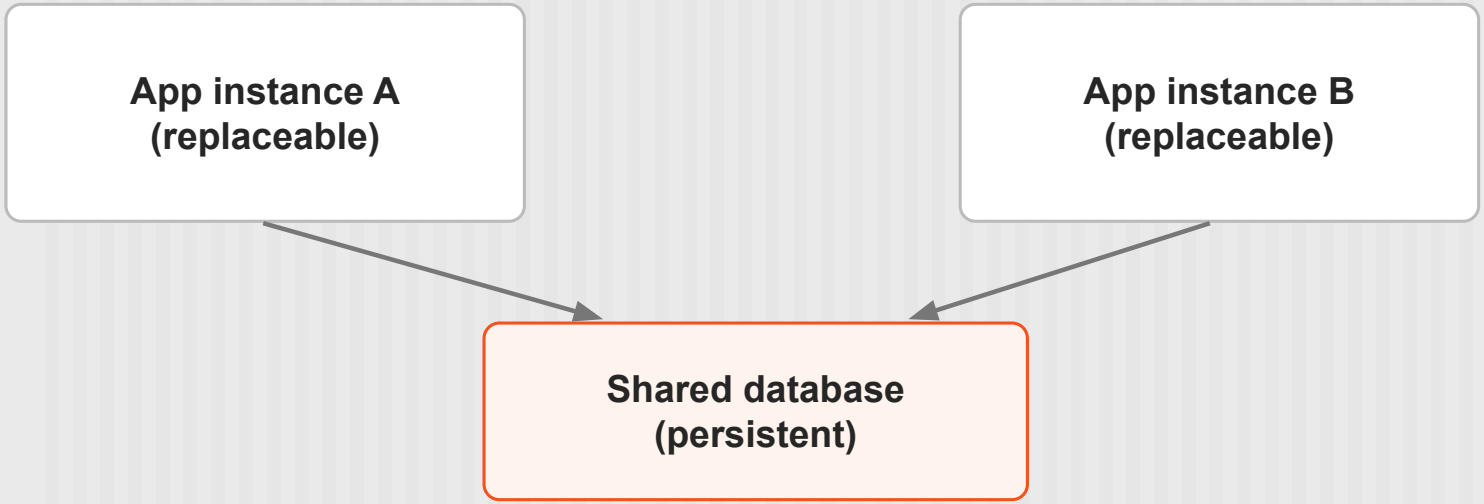
Each VM includes its own full guest operating system.

Container model



Containers share the host kernel. They package the app and its dependencies, not a full OS. Docker Desktop on Mac or Windows runs a Linux VM quietly in the background

Stateless app, persistent data



Question: If the app instance disappears on payday, must salary records disappear?

Stateless does not mean the application has no data. It means the application does not depend on data being stored locally within a particular running instance.

Payroll is busiest on payday

Vertical scaling

Give one server more CPU or memory.

Horizontal scaling

Add more application instances for traffic.

Load balancing

Distribute requests among healthy instances.

Reliability

Reduce single points of failure; monitor and recover.

PART B

The concepts behind your Lab 5 GCP deployment

What is inside a Google Cloud Project?

GCP Project

Billing · IAM · enabled APIs · region

Cloud Run
App runtime

Cloud SQL
Database

Artifact Registry
Images

Secret Manager
Credentials

Cloud Logging
Logs

Provision = create/configure a resource, e.g., a database instance.

IAM = controls access to cloud resources, while application authorization controls access to business functions.

Enabling an API allows the corresponding GCP service to be used.

Region is physical geography of service placement.

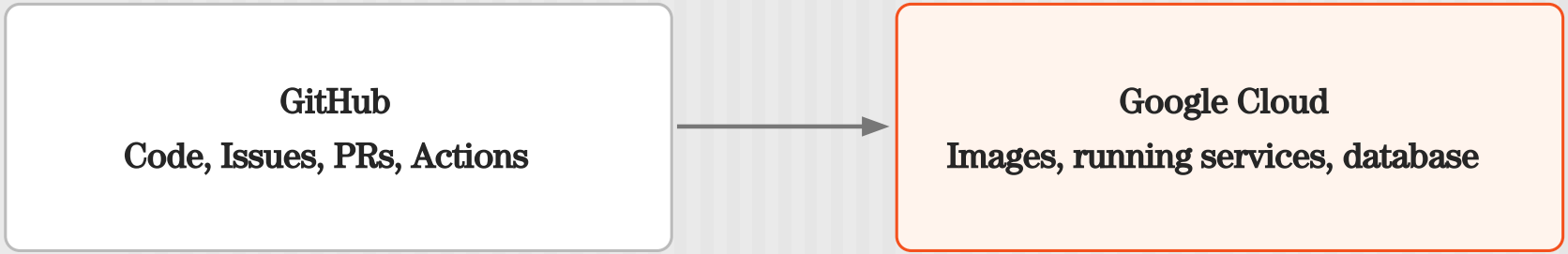
How Identity & Access Management (IAM) works

IAM has three main concepts:

1. Principal (Who): A person, group, or service account requesting access.
2. Role (What permissions): A collection of allowed actions, such as deploying a Cloud Run service.
3. Policy (Who gets which role): Specifies which principals receive roles on particular resources.

Identity (Who?)	Resource (What?)	Permission (Allowed action?)
Developer	Cloud Run	Deploy new application versions
Database administrator	Cloud SQL	Manage the database
CI/CD pipeline	Cloud Run	Deploy approved releases
Application service account	Cloud SQL	Connect to the database
Application service account	Secret Manager	Read specific application secrets

GitHub is not GCP



GitHub Actions authenticates to GCP to publish an approved container image and deploy it.

The live application does not run “inside GitHub.”

Docker: source code → image → running container

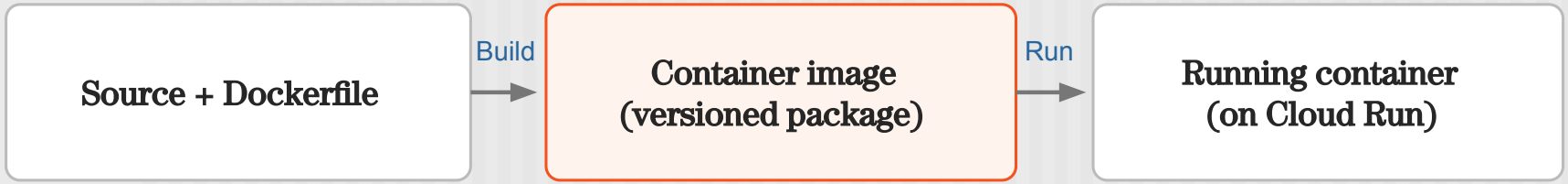


Image = packaged instructions and files.

Container = a running instance of that image.

Environment variables change configuration without rebuilding the image.

Can create the Docker image in GitHub for GCP

In Lab 5's TokTickIT CI/CD implementation, the Docker image is created in GitHub Actions and then uploaded to GCP Artifact Registry.

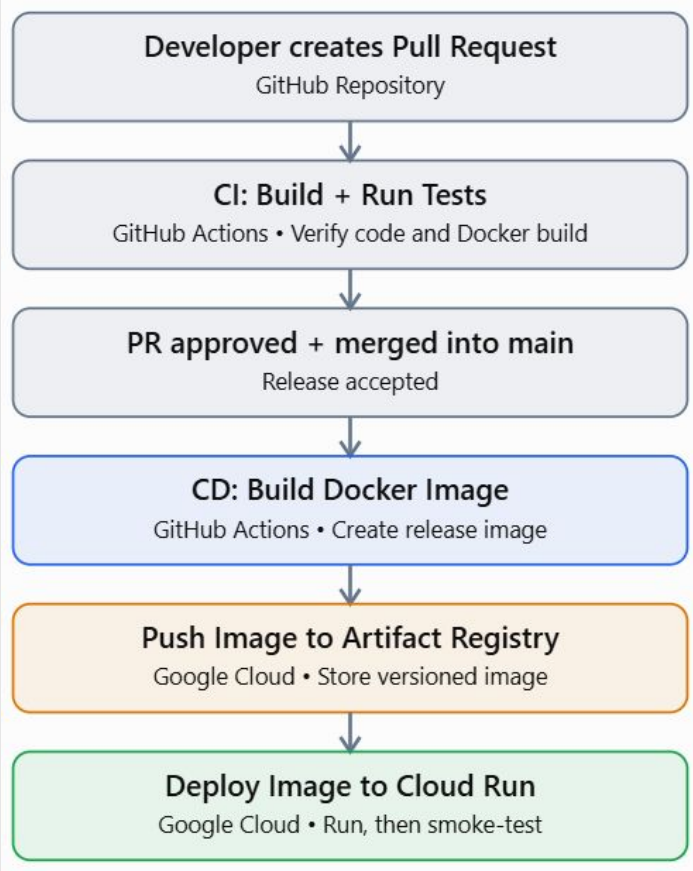
Alternative Approach Build on GCP:

GitHub Actions → Cloud Build → Artifact Registry → Cloud Run

CI asks: "Is this change safe to integrate?"

CD asks: "Can we package and deliver the approved version to users?"

And most importantly, Cloud Run runs the Docker image. It does not need to rebuild the source code.



What exactly is a release?

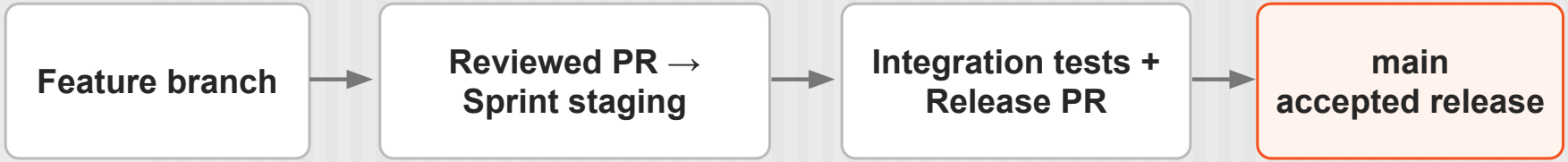


Traceability

We must know which source commit produced the deployed version, and which tests ran against it. When something breaks in production, we must be able to answer three questions: What version is running, what source code produced it, and what tests did it pass?

Identifier	What it identifies	Example
Git commit SHA	A particular Git commit	8f29a3c...
Docker image digest	The exact container image content	sha256:a1b2c3...
Cloud Run revision	A deployed version of the service configuration	payroll-00012-abc

How does a developer change reach main?



CI checks run on PRs. A failing required check must block the normal merge path.

The TAs will demonstrate a simplified release process by merging a Pull Request directly into main, so we can focus on the CI/CD pipeline and GCP deployment.

However, your Lab 5 requires our full course Git workflow. You must first merge your feature branch into the sprint staging branch through a reviewed PR, perform integration testing, and then create a release PR from staging into main.

Your teammate must approve the relevant PR, and required CI checks must pass. The checks must cover both integration and release PRs. Only then should the approved release be deployed to GCP.

Two automations, different responsibilities

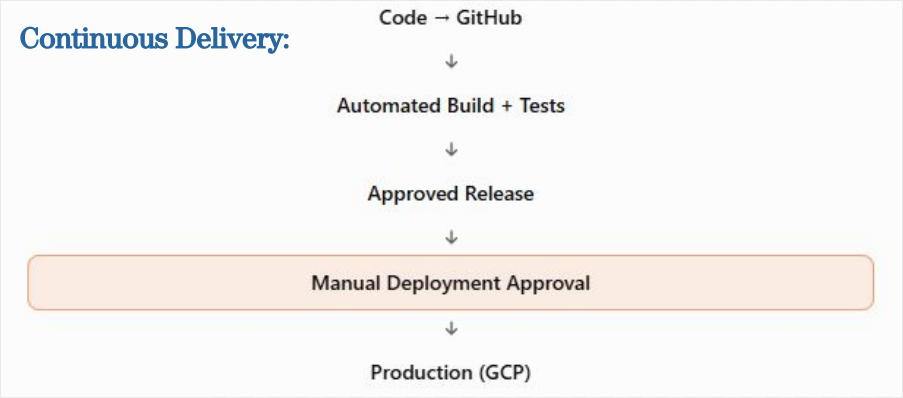
CI: Continuous Integration

Triggered by a PR or commit. Install dependencies, build, run tests, and report PASS/FAIL. Protect the codebase.

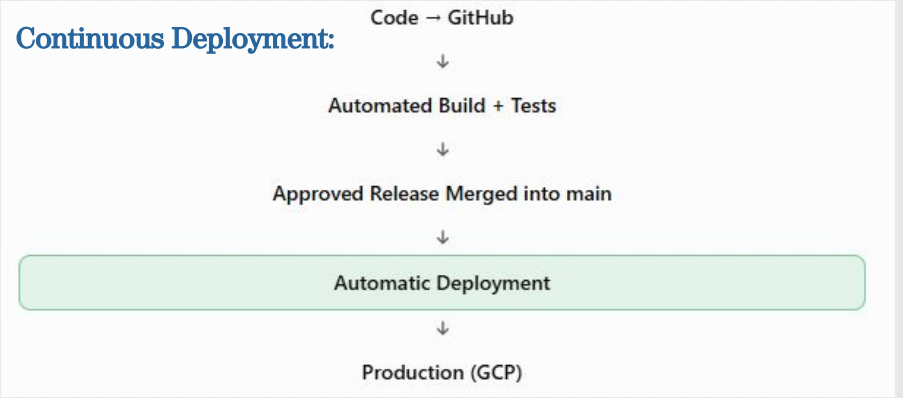
CD: Delivery / Deployment

Triggered by an approved release. Build/publish the image, migrate, deploy, then check the running service.

Continuous Delivery:



Continuous Deployment:



What actually runs CI/CD?

GitHub workflow YAML

A versioned file in .github/workflows defines triggers, jobs and steps.

Build and test commands

E.g., install packages, run unit tests and build the Docker image.

Deployment script

Versioned instructions create/update GCP services and deploy an image.

Authentication to GCP

Workload Identity Federation supplies short-lived credentials.

From accepted code to users

1. Merge to main
2. Run tests
3. Build image
4. Push registry
5. DB migrations
6. Deploy Cloud Run
7. Smoke tests

Two important checks

A build can pass while the deployed website fails.

A migration can change the database before deployment succeeds.

Smoke test ≠ full acceptance test

Smoke test: quick safety check

After deployment: website loads, API responds, database is reachable, critical route works.

UAT: user acceptance test

A real user verifies business scenarios: requester creates ticket; IT staff resolves; permissions hold.

Lab 5: smoke + persistence. Lab 6: independent UAT. Lab 7: regression after change.

Who can do what? Three different security layers

IAM

Controls which Google identities and service accounts can manage or access GCP resources.

Secrets

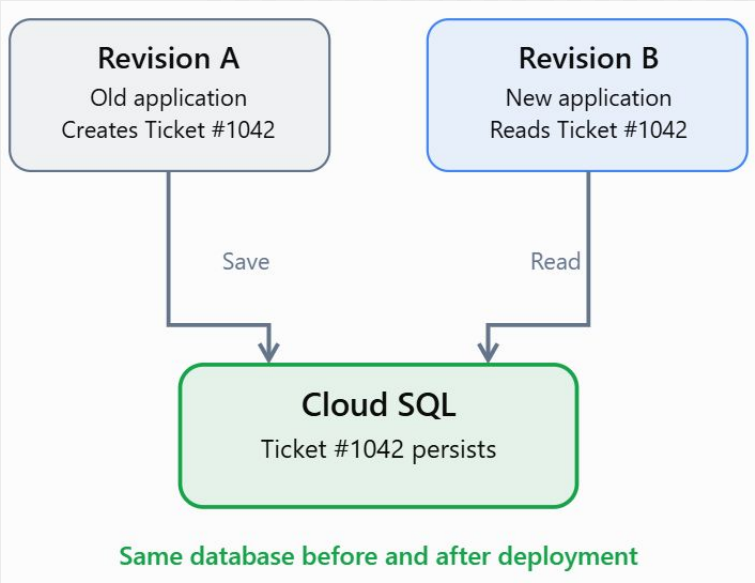
Passwords and credentials come from Secret Manager, not hard-coded source files.

- DB password
- LLM API key

App authorization

TokTickIT Requester and IT Staff roles control access to tickets, independent of GCP IAM.

Deploying a new application must not erase old tickets



What the TAs will demonstrate

1. Before deployment (Revision A): Log into TokTickIT, create Ticket #1042, attach a file, and record the ticket number.
2. Deploy Revision B: Merge an approved change into main. GitHub Actions builds and deploys the new application version to Cloud Run.
3. After deployment (Revision B): Open the same ticket and verify that its details and attachment are still intact.
4. After rollback: Redeploy the previous application image and verify that the ticket and attachment remain accessible.

Should verify that both Ticket #1042 and its attachments still exist in PostgreSQL.

What if the new payroll release is broken?

Rollback application

Redeploy a previously known-good container image or revision. Verify the running app again.

Database is different

An old application may not work with a new schema. Plan migrations and compatibility; rollback is not database restore.

Database rollback is much more complicated. Often use backward-compatible migrations.

Release ≠ “done forever.” Observe, verify and be ready to recover.

Security, availability and the bill

Security

Use HTTPS, least-privilege IAM, application permissions and secret management.

Reliability

Use logging, health checks, persistence, autoscaling and a recovery plan.

Cost

Compute, database, storage and network may all cost money. A budget alert is NOT a spending cap.

What you will actually build in the TA session

A CI and required quality gate

B Reproducible Docker build and configuration

C GCP application + database, durable data

D Repeatable reviewed release and smoke tests

Your task: adapt the reference stack to your Sprint 1 product, not simply copy the demo.

THE END

Next: TokTickIT deployment on Google Cloud with the TA team

つづく

tsuzuku